this behavior, this test would be slower than the unit tests you just wrote, and it would be harder to diagnose failures.

That said, you may be in a situation where there's value in writing the test strictly as an acceptance test, to verify behavior as part of a set of requirements rather than to drive development.

Prescription 32	Write your test to be as close as possible to the code logic that's being tested.

Using Roles

Now that you have the concept of users and roles in the system, you need to look at other places where users need access to a project. Two interesting places spring to mind:

- The project index list, where access should be limited to only the projects that the user can see
- The new tasks form, which should be limited to only the projects a user can see

Let's look at the index page. Two places need code here. A User instance needs some way to return the list of projects the user can see, and the controller index action needs to call that method. That argues for an integration test, though only weakly. Sometimes I'll skip an integration test if the logic is very close to Rails default integration and would easily be caught manually. Another option would be a request test that used test doubles to confirm that a specific method is called on User or Project:

security/04/spec/system/user_and_role_spec.rb

```
describe "index page" do
  let!(:my_project) { create(:project, name: "My Project") }
  let!(:not_my_project) { create(:project, name: "Not My Project") }

  it "allows users to see only projects that are visible" do
    my_project.roles.create(user: user)
    login_as(user)
    visit(projects_path)
    expect(page).to have_selector("#project_#{my_project.id}")
    expect(page).not_to have_selector("#project_#{not_my_project.id}")
  end

end
```

This test creates two projects, adding one to the user (the user is defined in a let statement earlier in the file). On visiting the index page, you expect to